Getting Started with Tkinter GUI for Python

Using Tkinter to Create Windows, Buttons and Labels

By **Dave Mallisk**

This document can help you get started with Tkinter, which is the most popular Graphical User Interface (GUI) package for the Python programming language. Tkinter automatically installs with Python. The example code in this document lets you create a new window that contains active buttons and dynamic labels. In the images above, red comment text, always preceded by #, shows how Tkinter widgets define buttons and labels.

What You Need

Before you can create and run the example program in this document, you need to download and install both Python 3 and IDLE for Python 3, which is its Integrated Development Environment (IDE). For more information, see Getting Started with Python Programming.

Creating an Example Tkinter GUI

To create an example GUI for Python through Tkinter, do the following:

1 - For each image above, click it to magnify it, copy it into a word-processor document, and then print that page.
2 - Start Python 3 and open a new window in IDLE.
3 - Type the code from the first image above, and then save the file with an appropriate name, such as TkinterExample.py.
4 - Run the module to verity that it creates a Button Test window similar to that in the first image above.
5 - Add the code from the second image above, and then save the file.
6 - Run the module to verify it adds green and blue buttons, and labels, similar to those in the second image above. Clicking the GREEN and BLUE buttons should populate the associated labels.
7 - Add the code from the third image above, and then save the file.
8 - Run the module to verify it adds and active reset button similar to that in the third image above. Clicking the Reset button should clear the label fields for the green and blue buttons. Verify also that the print() functions display correctly in the Python shell.

Copying the Example Code

To save time, you can click Tkinter Buttons and Labels, copy the example code, and then paste it into IDLE. For more information, see Running Python Examples.

Hopefully, this document has helped you get started with Tkinter for Python.

Sources

* - Python Info Wiki, "Tkinter", Python.org
* - John W. Shipman, "Tkinter 8.4 Reference: A GUI for Python", New Mexico Tech

More resources

- http://docs.python.org/release/3.0.1/library/tkinter.html

2011 © Associated Content, All rights reserved.
Privacy Policy | Terms of Service

How to Enter Data Through Python Tkinter.Ttk

Using Tkinter.Ttk Widgets to Create Data Entry Fields in Python

By **Dave Mallisk**

This document shows how to use Tkinter Themed Tk (ttk) widgets to program Python GUI data-entry interfaces for both text and numerical values. Tkinter ttk provides enhanced widgets that can simplify creating GUI windows.

What You Need

Before you can program and run ttk data-entry widgets, you need to download and install Python 3 and IDLE for Python 3. IDLE is the Python Integrated Development Environment (IDE). For details, see Getting Started with Python Programming.

Creating a ttk Planning Grid

Before you start programming your ttk GUI window, you should create a planning grid that helps you organize your Entry, Label and Button widgets into rows and columns. To create your ttk grid, do either of the following:

* - Make a pencil sketch.
* - Use a word-processor table as shown in the first image above.

Note: Compare the first three images above. The second shows the initial example window. The third shows the results of entering data into that window.

Programming Your ttk GUI Window
In the fourth and fifth images above, the example code contains ttk widgets. To create a similar program, do the following:
1 - At the start of your program, import both tkinter and ttk.
2. - Define callable GUI functions using the get() and set() control-variable methods. In the example code, the texter() function gets input data for variable 'textin,' and sets the output data for variable 'textout.' Also, the adder() function gets input data for variables 'a' and 'b,' and sets the output data for variable 'total.'
3 - Set up your GUI window. Title it, and define the grid so that your window can resize itself automatically as you add context.
4 - According to your ttk planning grid, create ttk widgets at appropriate rows and columns. The example code includes two ttk.Button widgets. The first (in row 3, column 3) uses 'command=texter' to call the texter() function. The second (in row 7, column 3) uses 'command=adder' to call the adder() function.
Duplicating the Example Code
To duplicate the example code in the fourth and fifth images above, you can either:
* - Click the fourth image to magnify it, copy it into a word-processor document, print that document, repeat for the fifth image, and then type the code into IDLE.
* - Click Tkinter ttk Data Entry, copy the code, and then paste it into IDLE. For details, see Running Python Examples.
Sources:
* - Python Standard Library, "tkinter.ttk - Tk Themed Widgets", Python.org
* - John W. Shipman, "Tkinter 8.4 Reference: Control Variables", New Mexico Tech

More resources

- http://www.tkdocs.com/index.html

How to Use Styles to Add Color to Python Ttk Widgets

Overriding Default Ttk Styles Enhances Python GUI Windows

By **Dave Mallisk**

Takeaways

- This document can help you enhance your Python GUI windows by applying styles to your ttk widgets. The example code in this document includes two programs that generate similar windows, one containing ttk widgets *without*styles, the other *with* styles.

  What You Need - Before you can use the example programs in this document, you need to download and install both Python 3 and IDLE for Python 3. IDLE is its Integrated Development Environment (IDE) for Python. For more information, see Getting Started with Python Programming.
  I developed these example programs by first creating a ttk planning grid as explained inHow to Enter Data Through Python Tkinter.Ttk.
  Ttk Widgets Without Styles -To generate the Ttk Widgets Without Styles window, do the following:
  1 - Click the first image above to magnify it, and then copy it into a word-processor document. Copy the second image above, paste it into the same document, and then print that document.
  2 - Start Python 3 and open a new window in IDLE.
  3 - Type the code from the first and second images above, and then save the file with an appropriate name, such as TtkStyles.py.
  4 - Run the module to verity it generates a Ttk Widgets Without Styles window similar to that in the third image above.
  Ttk Widgets With Styles - You can add styles by modifying the program in the section above. To generate the Ttk Widgets With Styles window, do the following:
  1 - Click the fourth image above to magnify it, copy it into a word-processor document, and then print that document.
  2 - As shown near the top of the fourth image above, add the 'ttk.Style().configure' style definitions. You can define a widget color as either a literal string in double quotes, such as "red", or as a hexadecimal string in single quotes, such as '#afd' in the Exit.Tbutton style. You can define a widget border relief as a "flat", "raised", "sunken", "groove" or "ridge" literal string in double quotes.
  3 - As shown near the bottom of the fourth image above, add 'style="ExitTButton"' to the Exit Program button widget.
  4 - Run the module to verity it generates a Ttk Widgets With Styles window similar to that in the fifth image above.
  Copying the Example Programs - As alternative to typing the example programs, you can copy them directly. As explained in Running Python Examples, you can click eitherTtk Widgets Without Styles orTtk Widgets With Styles, copy the code, and then paste it into IDLE.
  Sources:

* - Python v3.1.3 documentation,"Ttk Styling", Python Software Foundation
* - John W. Shipman, "Tkinter 8.4 Reference: A GUI for Python - Standard Attributes", New Mexico Tech

- This document can help you enhance your Python GUI windows by applying styles to your ttk widgets. The example code in this document includes two programs that generate similar windows, one containing ttk widgets *without*styles, the other *with* styles.

  What You Need - Before you can use the example programs in this document, you need to download and install both Python 3 and IDLE for Python 3. IDLE is its Integrated Development Environment (IDE) for Python. For more information, see Getting Started with Python Programming.
  I developed these example programs by first creating a ttk planning grid as explained inHow to Enter Data Through Python Tkinter.Ttk.
  Ttk Widgets Without Styles -To generate the Ttk Widgets Without Styles window, do the following:
  1 - Click the first image above to magnify it, and then copy it into a word-processor document. Copy the second image above, paste it into the same document, and then print that document.
  2 - Start Python 3 and open a new window in IDLE.
  3 - Type the code from the first and second images above, and then save the file with an appropriate name, such as TtkStyles.py.
  4 - Run the module to verity it generates a Ttk Widgets Without Styles window similar to that in the third image above.
  Ttk Widgets With Styles - You can add styles by modifying the program in the section above. To generate the Ttk Widgets With Styles window, do the following:
  1 - Click the fourth image above to magnify it, copy it into a word-processor document, and then print that document.
  2 - As shown near the top of the fourth image above, add the 'ttk.Style().configure' style definitions. You can define a widget color as either a literal string in double quotes, such as "red", or as a hexadecimal string in single quotes, such as '#afd' in the Exit.Tbutton style. You can define a widget border relief as a "flat", "raised", "sunken", "groove" or "ridge" literal string in double quotes.
  3 - As shown near the bottom of the fourth image above, add 'style="ExitTButton"' to the Exit Program button widget.
  4 - Run the module to verity it generates a Ttk Widgets With Styles window similar to that in the fifth image above.
  Copying the Example Programs - As alternative to typing the example programs, you can copy them directly. As explained in Running Python Examples, you can click eitherTtk Widgets Without Styles orTtk Widgets With Styles, copy the code, and then paste it into IDLE.
  Sources:
  * - Python v3.1.3 documentation,"Ttk Styling", Python Software Foundation
  * - John W. Shipman, "Tkinter 8.4 Reference: A GUI for Python - Standard Attributes", New Mexico Tech

- This document can help you enhance your Python GUI windows by applying styles to your ttk widgets. The example code in this document includes two programs that generate similar windows, one containing ttk widgets *without*styles, the other *with* styles.

  What You Need - Before you can use the example programs in this document, you need to download and install both Python 3 and IDLE for Python 3. IDLE is its Integrated Development Environment (IDE) for Python. For more information, see Getting Started with Python Programming.
  I developed these example programs by first creating a ttk planning grid as explained inHow to Enter Data Through Python Tkinter.Ttk.
  Ttk Widgets Without Styles -To generate the Ttk Widgets Without Styles window, do the following:
  1 - Click the first image above to magnify it, and then copy it into a word-processor document. Copy the second image above, paste it into the same document, and then print that document.
  2 - Start Python 3 and open a new window in IDLE.
  3 - Type the code from the first and second images above, and then save the file with an appropriate name, such as TtkStyles.py.
  4 - Run the module to verity it generates a Ttk Widgets Without Styles window similar to that in the third image above.
  Ttk Widgets With Styles - You can add styles by modifying the program in the section above. To generate the Ttk Widgets With Styles window, do the following:
  1 - Click the fourth image above to magnify it, copy it into a word-processor document, and then print that document.
  2 - As shown near the top of the fourth image above, add the 'ttk.Style().configure' style definitions. You can define a widget color as either a literal string in double quotes, such as "red", or as a hexadecimal string in single quotes, such as '#afd' in the Exit.Tbutton style. You can define a widget border relief as a "flat", "raised", "sunken", "groove" or "ridge" literal string in double quotes.
  3 - As shown near the bottom of the fourth image above, add 'style="ExitTButton"' to the Exit Program button widget.
  4 - Run the module to verity it generates a Ttk Widgets With Styles window similar to that in the fifth image above.

Copying the Example Programs - As alternative to typing the example programs, you can copy them directly. As explained in Running Python Examples, you can click eitherTtk Widgets Without Styles orTtk Widgets With Styles, copy the code, and then paste it into IDLE.

Sources:

* - Python v3.1.3 documentation,"Ttk Styling", Python Software Foundation
* - John W. Shipman, "Tkinter 8.4 Reference: A GUI for Python - Standard Attributes", New Mexico Tech

This document can help you enhance your Python GUI windows by applying styles to your ttk widgets. The example code in this document includes two programs that generate similar windows, one containing ttk widgets *without* styles, the other *with* styles.

What You Need - Before you can use the example programs in this document, you need to download and install both Python 3 and IDLE for Python 3. IDLE is its Integrated Development Environment (IDE) for Python. For more information, see Getting Started with Python Programming.

I developed these example programs by first creating a ttk planning grid as explained in How to Enter Data Through Python Tkinter.Ttk.

Ttk Widgets Without Styles - To generate the Ttk Widgets Without Styles window, do the following:

1 - Click the first image above to magnify it, and then copy it into a word-processor document. Copy the second image above, paste it into the same document, and then print that document.

2 - Start Python 3 and open a new window in IDLE.

3 - Type the code from the first and second images above, and then save the file with an appropriate name, such as TtkStyles.py.

4 - Run the module to verity it generates a Ttk Widgets Without Styles window similar to that in the third image above.

Ttk Widgets With Styles - You can add styles by modifying the program in the section above. To generate the Ttk Widgets With Styles window, do the following:

1 - Click the fourth image above to magnify it, copy it into a word-processor document, and then print that document.

2 - As shown near the top of the fourth image above, add the 'ttk.Style().configure' style definitions. You can define a widget color as either a literal string in double quotes, such as "red", or as a hexadecimal string in single quotes, such as '#afd' in the Exit.Tbutton style. You can define a widget border relief as a "flat", "raised", "sunken", "groove" or "ridge" literal string in double quotes.

3 - As shown near the bottom of the fourth image above, add 'style="ExitTButton"' to the Exit Program button widget.

4 - Run the module to verity it generates a Ttk Widgets With Styles window similar to that in the fifth image above.

Copying the Example Programs - As alternative to typing the example programs, you can copy them directly. As explained in Running Python Examples, you can click either Ttk Widgets Without Styles or Ttk Widgets With Styles, copy the code, and then paste it into IDLE.

Sources:

* - Python v3.1.3 documentation, "Ttk Styling", Python Software Foundation
* - John W. Shipman, "Tkinter 8.4 Reference: A GUI for Python - Standard Attributes", New Mexico Tech